

KATHMANDU UNIVERSITY
Department of Computer Science and Engineering

A Project Defense Presentation on

"Readify"

A Social Book Recommendation Platform using
Graph Attention Networks (GAT)

PREPARED BY

Saurav Maske(10)

Utsav Subedi(18)

Aarju Taujale (19)

Aakash Thakur (20)

Supervisor: Mr. Dhiraj Shrestha | Code:COMP313

3rd Year / 2nd Semester

Agendas

-
- **1. Introduction**
Background, objectives & motivation
 - **2. Related Works**
Existing platforms & GAT literature
 - **3. System Design**
Architecture, database & recommendation engine
 - **4. Requirements & Planning**
Specs, UI, timeline, conclusion
-

01

Introduction

Setting the stage: why Readify, and what it aims to achieve

Background & Problem Statement

Readers increasingly rely on online communities (Goodreads, StoryGraph, Hardcover) to discover books — but most recommendation engines behind them are limited.

- Cold-Start Problem

New users/books get poor recommendations

- Data Sparsity

Most user–book interactions are missing

- Popularity Bias

Only popular titles keep getting recommended

- Weak Relationship Modelling

Social & complex relations are ignored

Our Approach

Model users, books, ratings, reviews & social ties as a graph — then apply Graph Attention Networks (GAT) so the model learns which neighbours matter most.

Project Objectives

To design and develop an intelligent social book recommendation platform that delivers personalised recommendations using Graph Attention Networks while fostering an interactive reading community.

- Design & Develop

A web-based social book recommendation platform named Readify.

- Construct a Graph

Represent users, books, ratings, reviews, genres & social relationships.

- Build the Engine

Implement an attention-based recommendation engine using GAT.

- Evaluate & Compare

Assess the model against conventional recommendation approaches.

02

Existing Systems and Platforms

- Collaborative Filtering

Recommends based on similarity among users or items (user-based / item-based).

LIMITATION

Cold-start, sparse matrices, popularity bias

- Content-Based Filtering

Uses genre, author, keywords & description similarity to prior likes.

LIMITATION

Limited diversity — repeats similar attributes

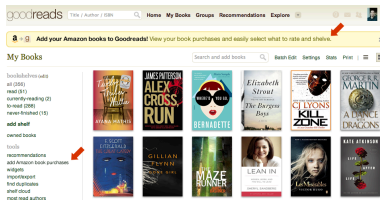
- Hybrid Systems

Combine collaborative & content-based methods to offset individual weaknesses.

LIMITATION

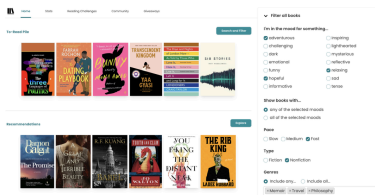
Still struggles with complex social relationships

Existing Book Recommendation Platforms



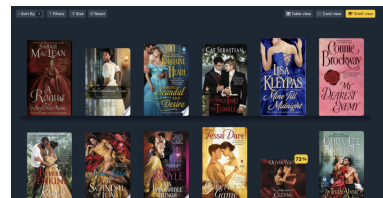
Goodreads

Collaborative filtering + interaction history; static, no graph learning.



StoryGraph

Mood/pace/theme metadata-driven; no deep graph model.



Hardcover

Strong social features; traditional recommendation logic only.

Graph Neural Networks & Graph Attention

GNNs learn from both node features AND relationships between nodes — ideal when users, books, authors, genres & reviews naturally form a graph.

- GAT (Veličković et al., 2018) assigns unequal importance to neighbouring nodes via an attention mechanism.
- Instead of treating every neighbour equally, GAT learns attention coefficients per edge.
- This lets the model focus on highly relevant interactions and suppress noisy ones.
- Applied successfully in recommendation, fraud detection, social & biological network analysis.

Comparison of Existing Systems & Research Gap

System	Social	AI Recs	Graph Learning	Major Limitation
Goodreads	Yes	Yes	No	Popularity bias, traditional methods
StoryGraph	Yes	Partial	No	Limited use of social relationships
Hardcover	Yes	Partial	No	Comparatively simple recommender
Readify	Yes	Yes	Yes (GAT)	

- Research Gap

Existing platforms rarely use adaptive attention to identify influential readers, so recommendations overlook trusted reviewer signals and indirect community interactions. Readify closes this gap by applying GAT over a unified user–book social graph.

03

System Design & Methodology

Development Methodology

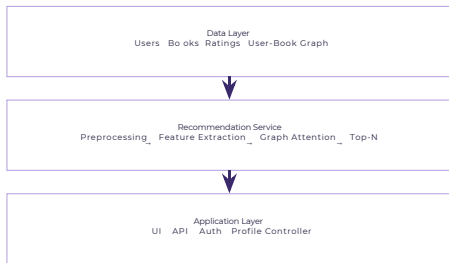
Readify followed an iterative software development methodology, continuously refined across phases:



Three-Tier Architecture

Presentation (React.js) → Application (Node.js/Express) → Data (PostgreSQL) — with a Python/PyTorch recommendation service alongside the backend.

Overall System Architecture



Layers

Data Layer

Users, books, ratings, user-book graph

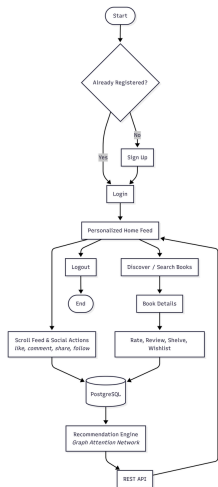
Recommendation Service

Preprocessing → features → graph attention → Top-N

Application Layer

UI, API, Auth, profile controller

System Workflow



- User registers/logs in (OTP or Google OAuth). Browses,
- searches, rates, reviews, shelves & follows readers.
- Every interaction is written to PostgreSQL. Interactions
- continuously update the user-book graph. GAT
- aggregates neighbour info & computes attention
- weights.
- Top-N recommendations return via the REST API to the home feed.

Technology Stack

•
Frontend

React.js

•
Backend

Node.js + Express.js

•
Database

PostgreSQL

•
Authentication

JWT + Google OAuth

Recommendation
Engine

Python

ML
Framework

PyTorch + PyG

Version
Control

Git & GitHub

Design &
Testing

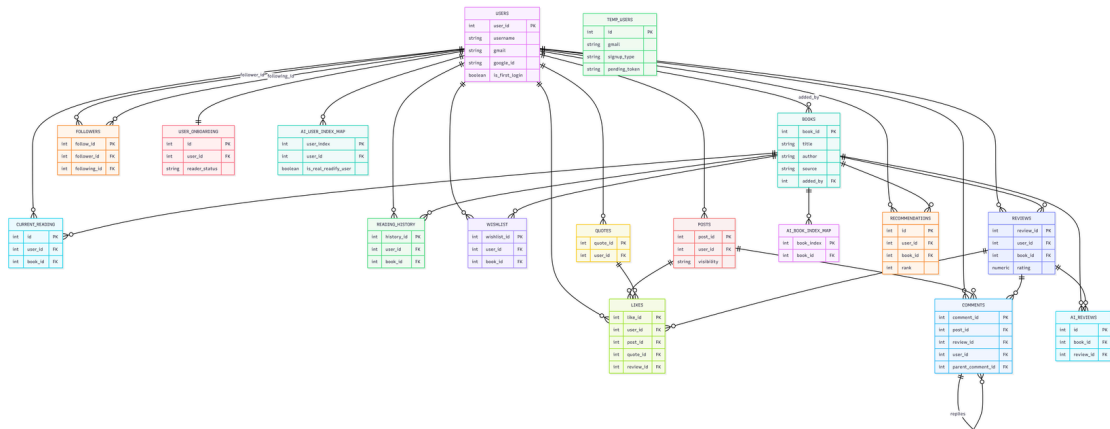
Figma + Postman

Database Design

Relational schema in PostgreSQL — Users, Books, Authors, Genres, Ratings, Reviews, Followers, Reading Lists & Recommendations, linked via primary/foreign keys.

- Users
- Books
- Authors
- Genres
- Ratings
- Reviews
- Followers
- Reading Lists
- Recommendations

Database Design



Recommendation Engine & Pipeline

The graph combines User–Book interactions, User–User follows, reviews, genres, authors & ratings. Nodes carry feature vectors; Graph Attention layers aggregate neighbour information with multiple attention heads for robustness.

Multiple attention heads improve representation learning and recommendation robustness.

Recommendation Pipeline:

1. User performs interactions
2. Interaction data stored in PostgreSQL
3. Graph data constructed
4. Node features generated
5. GAT computes embeddings
6. Compare user & candidate book embeddings
7. Top-N recommendations generated
8. Recommendations displayed on homepage

Authentication Process

Registration → Verification (OTP / OAuth) → Password Encrypted → JWT Issued → Session Managed

- ## OTP Verification

Manual registration verified via one-time password.

- ## Google OAuth 2.0

One-click sign-in delegating identity verification.

- ## Bcrypt Encryption

Passwords securely hashed before storage.

- ## JWT Sessions

Tokens issued to manage sessions securely.

04

Requirements & Conclusions

Functional & Non-Functional Requirements

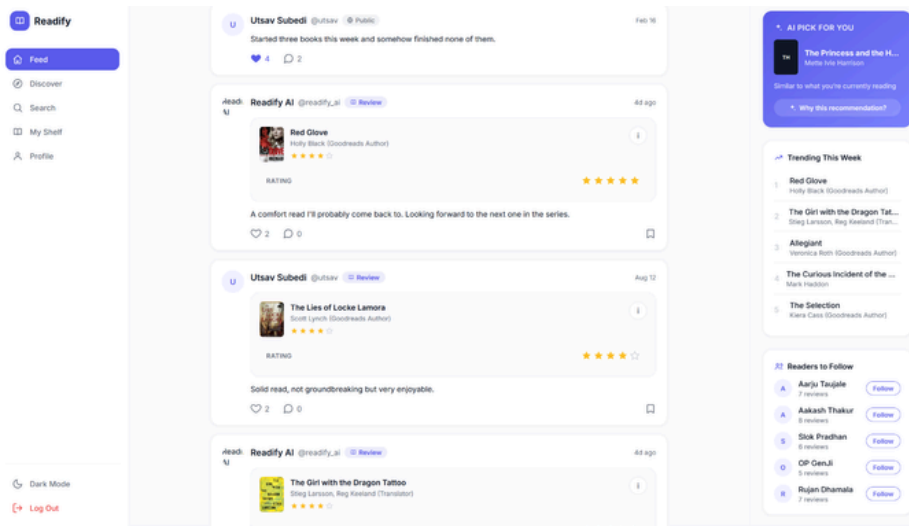
FUNCTIONAL

- Registration (manual + Google Sign-In) & secure authentication
- Book search, browsing & detailed book pages
- 5-star ratings and full review system
- Multiple bookshelves — Reading, Completed, Wishlist, Custom
- Social features: follow, likes, comments, activity feed GAT-based personalised recommendations & notifications
- Admin controls for users, books & reported content

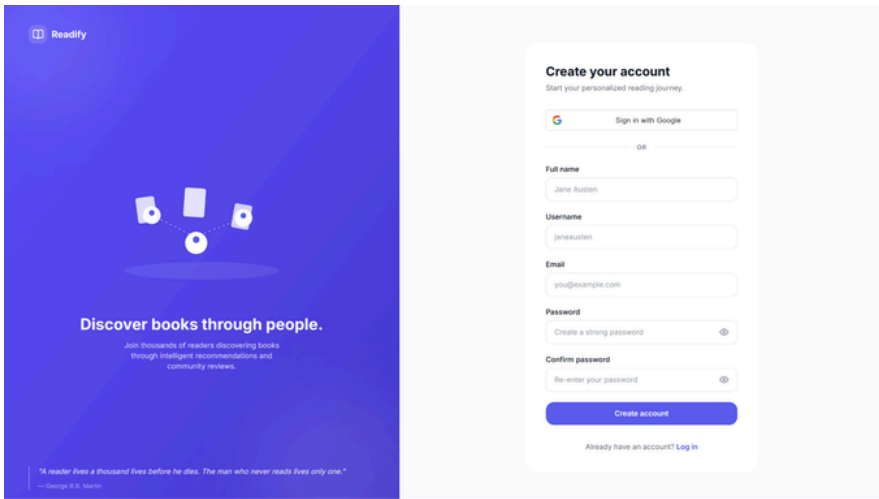
NON-FUNCTIONAL

- High availability & fast recommendation response
- Secure authentication and authorization
- Scalable, RESTful backend architecture
- Responsive, cross-platform user interface
- Reliable database transactions
- Secure password encryption
- Easy maintenance & future extensibility

User Interface Design



User Interface Design



Conclusion & Limitations

CONCLUSION

Readify combines social networking with Graph Attention Networks to model users, books, ratings, reviews and social ties as a connected graph — capturing complex relationships that conventional collaborative or content-based filtering miss, while encouraging real community engagement.

LIMITATIONS

- Recommendation quality depends on sufficient interaction data
GAT training requires GPU
- resources for efficiency New users get limited recommendations until history builds up
- Model evaluated on a limited dataset.

Future Work

- LLM Integration

Conversational, natural-language book recommendations.

- Mobile App

Native experience built with Flutter.

- Cloud-Scale Deployment

Production-grade infrastructure with multimodal recommendation.

Thank You!

Questions & Discussion

Saurav Maske • Utsav Subedi • Aarju Taujale • Aakash Thakur

Department of Computer Science and Engineering
Kathmandu University